
Documentazione Progetto Software Architecture Design

Educational Game for Man vs Automated Testing Tools challenges

Team D11

Studenti:

Amarante Matteo [mat.amarante@studenti.unina.it]

Di Costanzo Mariagrazia [mariagr.dicostanzo@studenti.unina.it]

Polisi Carmine [car.polisi@studenti.unina.it]

Prof.ssa:

Anna Rita Fasolino

Contents

Introduzione	3
1 Analisi	5
1.1 Analisi dei Requisiti	5
Analisi dei Requisiti	5
1.2 Use Case Diagram	7
1.2.1 UC-01: showProfileNotificationPage	8
1.2.2 UC-02: getNotifications	9
1.2.3 UC-03 : sendNotification	10
1.3 Class Diagram	11
1.4 Context Diagram	12
2 Gestione delle Notifiche : proposta del team	14
2.0.1 RabbitMQ	14
2.1 Pattern scelti	15
2.2 Component Diagram	16
2.3 Component Diagram v2	17
3 Progettazione	19
3.1 Progettazione	19
Progettazione	19
3.2 Class Diagram	20
3.3 Sequence Diagram	21
3.4 Anti-Pattern	24
4 Implementazione	25
Modifiche effettuate	25
4.1 Creazione di un nuovo servizio nel modulo T5	27
4.2 Deployment Diagram	29
4.3 Problematiche riscontrate	31
4.3.1 Risultato finale	31

4.4	Recap: Moduli modificati	33
5	Testing	35
5.1	Test funzionale	35
6	Conclusioni	42
	Conclusioni	42
6.1	Risultati Raggiunti	42
6.1.1	Scelte progettuali	43
6.1.2	Sviluppi Futuri	43
6.1.3	Considerazioni Finali	44

Introduzione

In questo progetto ci è stato proposto di lavorare su un'architettura reale, organizzata a microservizi, utilizzando un approccio basato sul gioco. Il task da sviluppare è l'R7 che richiede l'introduzione di un sistema generalizzato di notifiche da mostrare nella home page del giocatore. Gli obiettivi fissati sono:

- Progettare un sistema flessibile che permetta all'applicazione di inviare notifiche tra diversi microservizi
- Mostrare le notifiche in modo chiaro e immediato nella Home del giocatore

Nello sviluppo del progetto abbiamo adottato una metodologia Agile iterativa, ispirata a Scrum, ma adattata alle dimensioni del nostro team. Ogni settimana abbiamo svolto un meeting di lunga durata, durante il quale:

- abbiamo verificato i progressi della settimana precedente
- abbiamo ridefinito i task prioritari
- abbiamo distribuito le attività tra sviluppo e progettazione

Il processo è stato quindi ciclico, incrementale e basato su continui momenti di revisione.

Durante la prima view è stato analizzato il codice dell'applicazione presentato, raccogliendo le informazioni appropriate allo sviluppo del task :

- Modulo T5: è uno dei microservizi più centrali dell'intera applicazione, perché presenta la UI del gioco e l'intero motore di gestione delle partite. Si occupa di gestire tutto ciò che riguarda l'esperienza di gioco, dalle schermate della web app alla logica della partita. è il controller centrale che coordina gli altri microservizi durante la partita.
- Modulo T23 : è il microservizio che gestisce registrazione, login, profili, autenticazione e dati associati ai giocatori. In pratica è il User Service del sistema.

-
- Comunicazione : lo scopo è ottenere dati del profilo e verificare l'utente.

Alla fine della prima review abbiamo ,analizzando i product backlog creati a partire dai requisiti, scelto quale soluzione adottare e la strategia migliore per soddisfare quest'ultimi.

Durante il secondo Sprint abbiamo selezionato delle voci dai vari backlog e fissato lo Sprint Goal per la suddetta iterazione. In particolare abbiamo analizzato la gestione degli errori e la categorizzazione delle code.

Si è poi proseguito con l'istanziamento del nuovo container per l'implementazione del RabbitMQ manager e il vero e proprio Broker per la gestione di exchange e code, collegandolo al resto dell'architettura dopo aver configurato i profili di rete e le dipendenze.

Si sono effettuate le modifiche ai moduli T23 per quanto riguarda i consumatori e il T5 per i produttori e modificato anche qui `application.properties` e dipendenze per far sì che la nuova infrastruttura asincrona fosse funzionante.

Il terzo ed ultimo Sprint ha avuto come Sprint Goal quello di implementare le funzionalità restanti e fornire un semplice esempio di funzionamento come richiesto dallo Stakeholder.

Si è poi proseguito con i vari casi di test e la produzione di un tutorial per poter rendere il tutto utilizzabile facilmente anche ai prossimi sviluppatori.

Chapter 1

Analisi

1.1 Analisi dei Requisiti

In prima analisi abbiamo identificato i requisiti del sistema. Rilevando requisiti funzionali e non funzionali. Ricordiamo i primi descrivere *cosa* il sistema deve fare, le azioni, le funzionalità e i servizi che il software deve fornire. I secondi, invece, descrivono *come* il sistema deve comportarsi, rappresentando lgi attributi di qualità del software.

Requisiti Funzionali

ID	Descrizione
RF1	Il sistema deve essere in grado di gestire notifiche di vario tipo.
RF2	Il sistema deve supportare la persistenza delle notifiche.
RF3	Il sistema deve consentire la gestione delle notifiche provenienti da diverse componenti dell'architettura.
RF4	Il sistema deve permettere l'associazione delle notifiche a uno specifico utente.

Table 1.1: Requisiti funzionali del sistema

Requisiti Non Funzionali

ID	Descrizione
RNF1	Il sistema deve essere in grado di gestire un elevato numero di eventi.
RNF2	Il sistema non deve perdere le notifiche generate.
RNF3	Le operazioni sulle notifiche devono essere facilmente estendibili.
RNF4	La latenza di invio e ricezione dei messaggi tra i moduli deve essere ridotta.

Table 1.2: Requisiti non funzionali del sistema

1.2 Use Case Diagram

Come prima rappresentazione si è deciso di realizzare un diagramma dei casi d'uso. In particolare, un caso d'uso è una tipica interazione tra un attore ed il sistema per svolgere un'unità di lavoro utile. L'insieme dei casi d'uso rappresenta le funzionalità del sistema. Adottando un approccio più generico, in prima analisi, si è scelto di rappresentare come attori i due moduli coinvolti nello sviluppo e di riportare le singole funzionalità che interessano il nostro task.

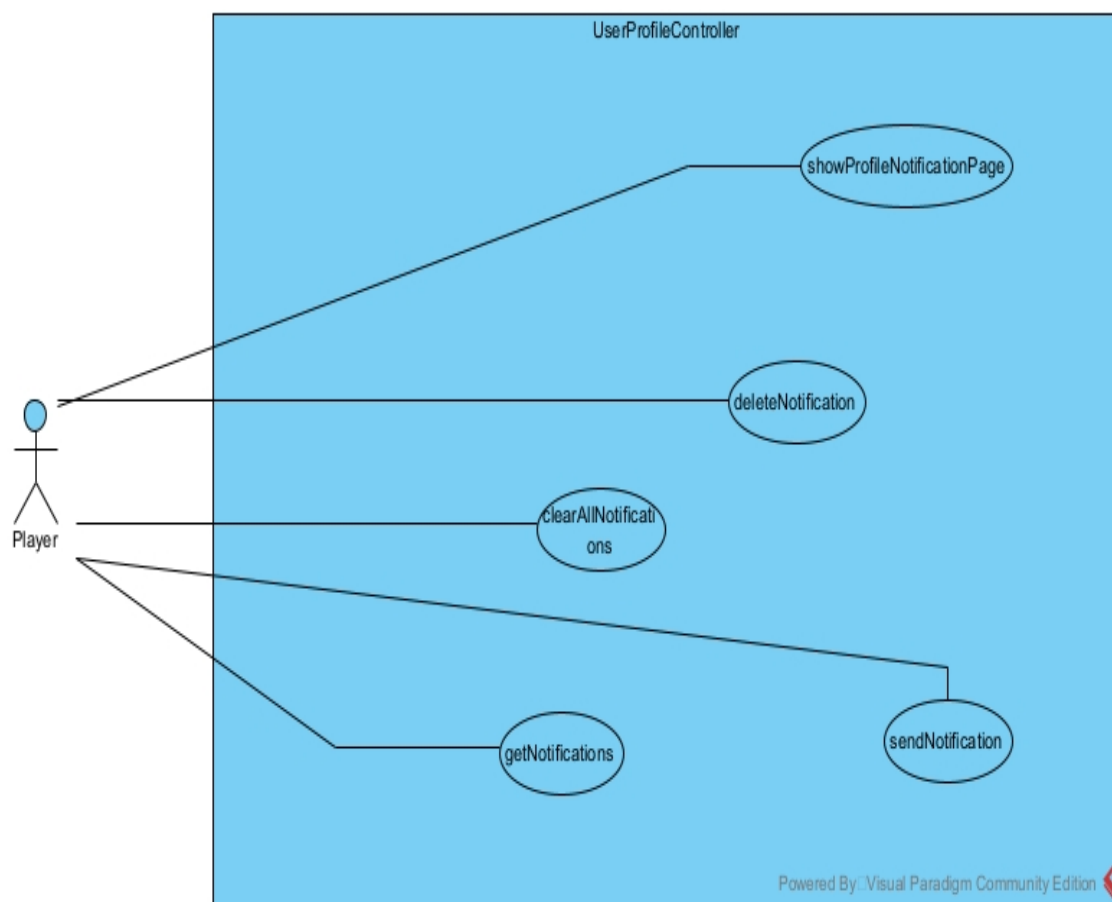


Figure 1.1: Diagramma dei Casi d'Uso per la classe `UserProfileController` del modulo T5

Ci teniamo a specificare che tutti i metodi sono stati aggiunti ex-novo. In particolare, il metodo `showProfileNotificationPage` è stato solo modificato. Di seguito proponiamo i scenari di alcuni casi d'uso:

1.2.1 UC-01: showProfileNotificationPage

Nome	showProfileNotificationPage
Attore Pri- mario	Player (Giocatore)
Scopo	Permettere al giocatore di visualizzare l'elenco delle proprie notifiche con paginazione
Precondizioni	• Il giocatore è autenticato nel sistema • Il giocatore ha un profilo valido nel database
Postcondizioni	• Le notifiche vengono visualizzate nella pagina • Il contatore delle notifiche viene aggiornato • Le notifiche lette sono distinte da quelle non lette
Descrizione	• L'utente richiede la visualizzazione delle proprie notifiche specificando la propria email • Il sistema prepara la richiesta • Il sistema effettua la richiesta • Il sistema verifica l'esistenza del profilo utente tramite email • Il sistema recupera le notifiche • Il sistema restituisce un oggetto contenente la lista paginata delle notifiche • L'utente visualizza le notifiche richieste
Flusso alterna- tivo	• nessuna notifica presente

Table 1.3: Scenario : showProfileNotificationPage

1.2.2 UC-02: getNotifications

Nome	getNotifications
Attore Pri- mario	Player (Giocatore)
Scopo	Consentire al player di raccogliere l'elenco delle proprie notifiche
Precondizioni	• Il giocatore è autenticato nel sistema • Il giocatore ha un profilo valido nel database
Postcondizioni	• Le notifiche vengono visualizzate nella pagina
Descrizione	• il Player invia una richiesta GET per recuperare le notifiche associate alla propria email • Il sistema prepara la richiesta • Il servizio di notifica pubblica un messaggio sul broker per l'elaborazione asincrona • Il sistema verifica l'esistenza del profilo utente tramite email • Il sistema recupera le notifiche • Il sistema restituisce un oggetto contenente la lista paginata delle notifiche • L'utente ottiene la lista di notifiche
Flusso alternativo	• Errore di comunicazione con RabbitMQ • l'email non è associata a nessun profilo

Table 1.4: Scenario : getNotifications

1.2.3 UC-03 : sendNotification

Nome	sendNotification
Attore Pri- mario	Player (Giocatore)
Scopo	Consente la creazione e il salvataggio di una nuova notifica per un utente
Precondizioni	• Il giocatore è autenticato nel sistema • Il giocatore ha un profilo valido nel database
Postcondizioni	• la notifica viene salvata nel db ed è disponibile per la consultazione
Descrizione	• Il player invia una richiesta di POST contenente email, oggetto e messaggio della notifica • Il sistema prepara la richiesta • Il servizio di notifica pubblica un messaggio su RabbitMQ per l'elaborazione asincrona • Il sistema verifica l'esistenza del profilo utente tramite email • Il sistema crea la notifica • Il sistema salva la nuova notifica • L'utente visualizza un messaggio di conferma
Flusso alterna- tivo	• l'email non è associata a nessun profilo • se il salvataggio fallisce, la notifica non viene persistita

Table 1.5: Scenario : sendNotification

1.3 Class Diagram

Successivamente abbiamo realizzato il diagramma delle classi in fase di analisi. Un diagramma delle classi mostra le classi del sistema e le relazioni tra di esse, includendo classi, attributi e metodi. Le relazioni possibili tra classi sono:

- Associazione : una linea continua, rappresenta un semplice collegamento tra classi.
- Dipendenza : una linea tratteggiata, per indicare una relazione d'uso
- Generalizzazione-specializzazione: una linea con freccia vuota, rappresenta la relazione di ereditarietà
- Aggregazione : forma di associazione lasca per indicare che la parte può esistere senza il tutto
- Composizione: forma di associazione forte per indicare che la parte non esiste se non col tutto.

In fase di analisi aiuta a capire quali concetti essitono nel sistema e come interagiscono. Mentre, in fase di progettazione serve a dettagliare le classi concrete.

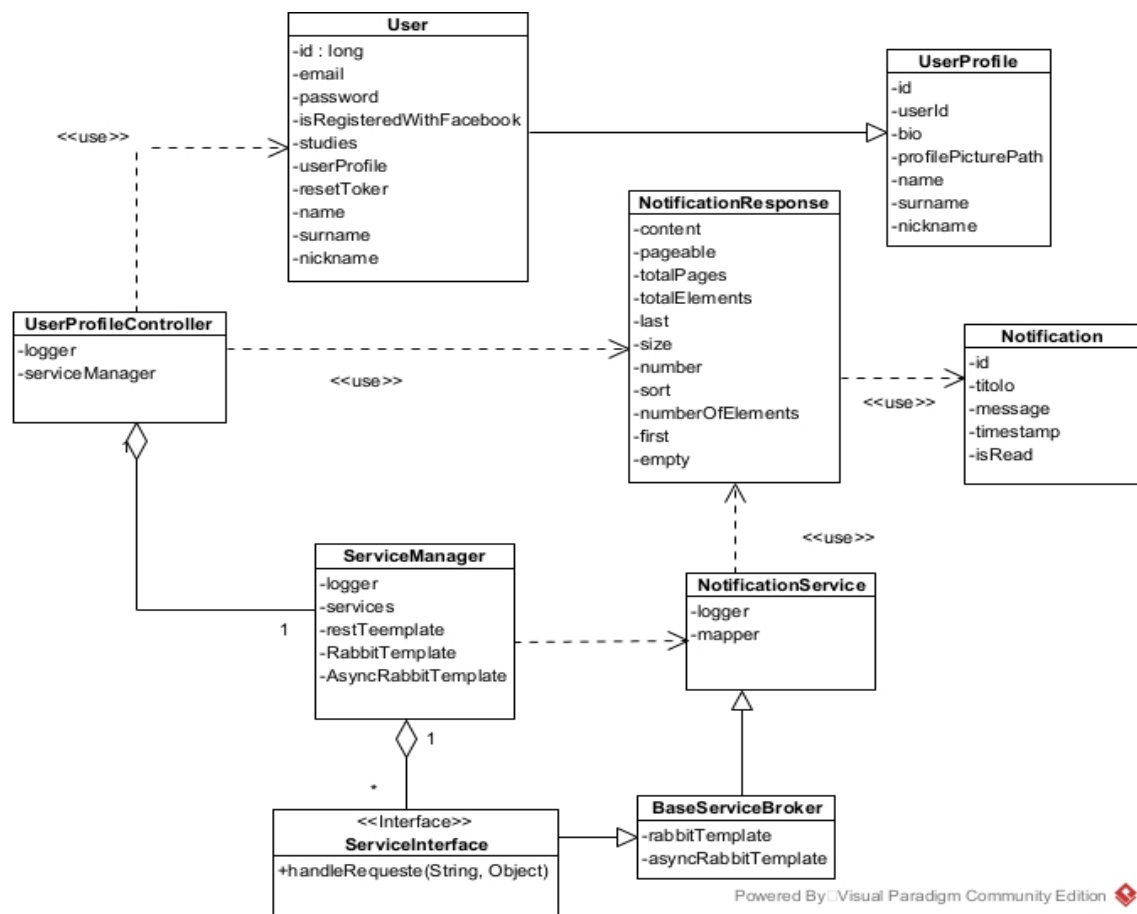


Figure 1.2: Class Diagram in fase di analisi per il modulo T5

1.4 Context Diagram

Il diagramma di contesto è uno degli strumenti più utili per rappresentare il sistema a un livello alto, senza entrare nei dettagli dell'implementazione in questa prima parte di analisi. Mostra il sistema come un unico blocco e tutti gli attori esterni con cui interagisce. Serve a capire chi interagisce con il sistema e quali informazioni o servizi vengono scambiati. I componenti sono:

- Sistema centrale : rappresentato da un rettangolo o cerchio grande. Può essere il sistema stesso o il microservizio principale
- Attori esterni : persone o altri microservizi. Sono collegati al sistema centrale con linee che mostrano il flusso di informazioni
- Flusso di dati : linee etichettate che indicano cosa viene scambiato tra attore

e sistema

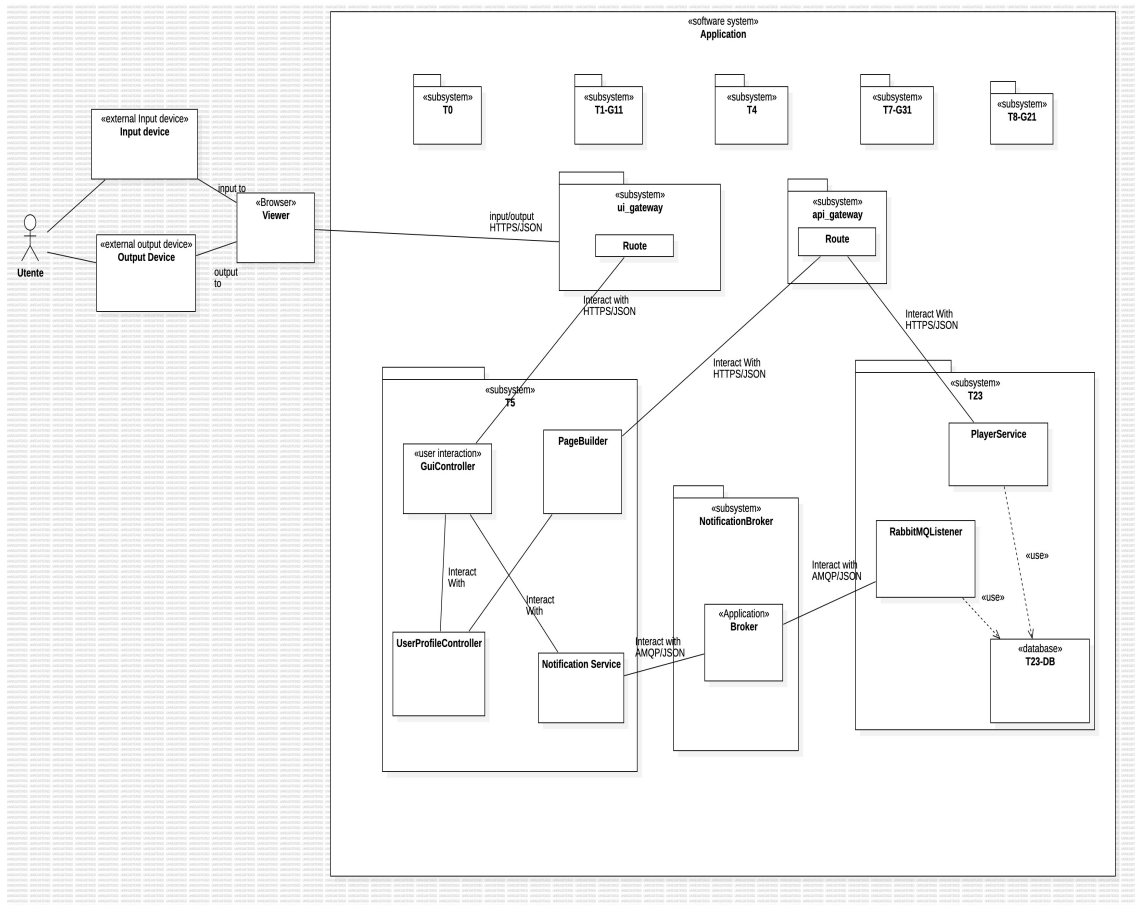


Figure 1.3: Context Diagram

Chapter 2

Gestione delle Notifiche : proposta del team

Proposta

La proposta del team riguarda l'introduzione di un broker intermedio tra i moduli che ne gestisca la comunicazione asincrona. Analizzando il codice dell'applicazione proposta, si è potuto notare la presenza di chiamate REST sincrone tra i vari moduli, rilevando un'alta latenza. Gli obiettivi del progetto riguardano l'implementazione di un sistema di notifiche generalizzato e riutilizzabile. Inoltre, la sostituzione di una comunicazione sincrona con una asincrona. Infine, l'assicurazione di persistenza e affidabilità nella gestione delle notifiche. Il broker introdotto è RabbitMQ.

2.0.1 RabbitMQ

Il funzionamento del broker riguarda la comunicazione tra producer, rappresentato dal modulo T5, e consumer, rappresentato dal modulo T23. Tale comunicazione non avviene in modo diretto, ma è mediata da una serie di componenti tipici dei sistemi di messagistica basati su broker, che garantiscono disaccoppiamento, affidabilità e scalabilità. Quando il modulo T5 genera una nuova notifica, esso agisce come producer e invia il messaggio all'exchange del broker. L'exchange ha il compito di ricevere il messaggio e instradarlo verso una o più code, in base alle regole di routing definite. È importante notare che l'exchange non conserva messaggi: la sua funzione è unicamente quella di determinare *dove* devono essere inviati. A valle dell'exchange si trovano le code, che rappresentano i veri punti di accumulo dei messaggi. Una coda può essere letta da uno o più consumer, i quali recuperano i messaggi in ordine FIFO e li elaborano. Nel nostro caso, il modulo T23 è il consumer che resta in ascolto sulla propria coda per ricevere le notifiche e

processarle. Affinché un exchange possa inoltrare correttamente i messaggi a una coda, è necessario definire un binding, ossia un collegamento logico che stabilisce la relazione tra l'exchange e la coda, spesso accompagnato da una routing key o da criteri di filtraggio specifici. Senza questo binding, l'exchange non sarebbe in grado di stabilire a quale coda consegnare il messaggio. Un aspetto importante riguarda la gestione dei casi in cui nessuna coda risulti collegata all'exchange. In questa situazione, i messaggi inviati dal producer vengono semplicemente scartati, poiché l'exchange non ha destinazioni valide a cui consegnarli. Questo comportamento, sebbene possa sembrare una perdita di informazione, è coerente con l'idea che l'assenza di binding indichi che non esistono consumer interessati a quei messaggi. Se nessun modulo T23 è in ascolto, non avrebbe senso mantenere notifiche inavese o accumularle in modo indefinito.

Grazie a questo modello, il broker permette una comunicazione affidabile e asincrona tra T5 e T23, garantendo che i moduli siano completamente disaccoppiati-il producer non deve conoscere il consumer, né preoccuparsi del suo stato-e che i messaggi vengano elaborati solo quando esiste un componente effettivamente interessato a riceverli.

2.1 Pattern scelti

- Fire-and-Forget: patter di comunicazione per operazioni senza risposta, come ad esempio `newNotification`, `deleteNotification`...
- RPC con code Temporanee: patter di comunicazione per operazioni con risposta, come ad esempio `getNotification`. La sua implementazione si descrive in passi:
 1. i. Il producer crea la coda temporanea con nome univoco;
 2. ii. Il producer invia messaggio con property `replyto`;
 3. iii. Il consumer processa e pubblica la risposta sulla coda indicata precedentemente;
 4. iv. Il producer riceve la risposta e cancella la coda temporanea.

Analizzando il codice, si è potuto osservare come i metodi implementati nella classe `T23Service` del modulo T5, siano categorizzati in metodi che restituiscono una semplice stringa di conferma (`newNotification/deleteNotification`...) e metodi che restituiscono un oggetto effettivo (`getNotification`)

2.2 Component Diagram

Questa soluzione può essere rappresentata dal diagramma dei componenti. Tale è un diagramma strutturale UML che modella l'organizzazione e le dipendenze tra i componenti fisici di un sistema software. Lo scopo principale è definire l'architettura generale del sistema, mostrando come le sue parti, i componenti, sono cablate tra loro e come interagiscono.



Figure 2.1: Component Diagram generale

Gli elementi principali sono:

- component : è una black box il cui comportamento è definito dalle sue interfacce
- interfaccia fornita (lollipop) : definisce i servizi che il componente offre al mondo esterno
- interfaccia richiesta (socket) : definisce i servizi che il componente necessita da un altro componente per funzionare
- port : un punto di interazione specifico tra un componente e il suo ambiente

Si pone in osservazione la possibilità di inserire in questo diagramma sia interfacce reali sia interfacce ideali.

- Le prime sono specifiche della tecnologia e dell'ambiente di esecuzione.
- Le seconde, invece, sono definite durante la fase di analisi e progettazione astratta. Sono contratti che specificano il comportamento o il servizio necessario, indipendentemente dalla tecnologia che verrà utilizzata per realizzarlo

2.3 Component Diagram v2

Di seguito un nuovo diagramma dei componenti più specifico :

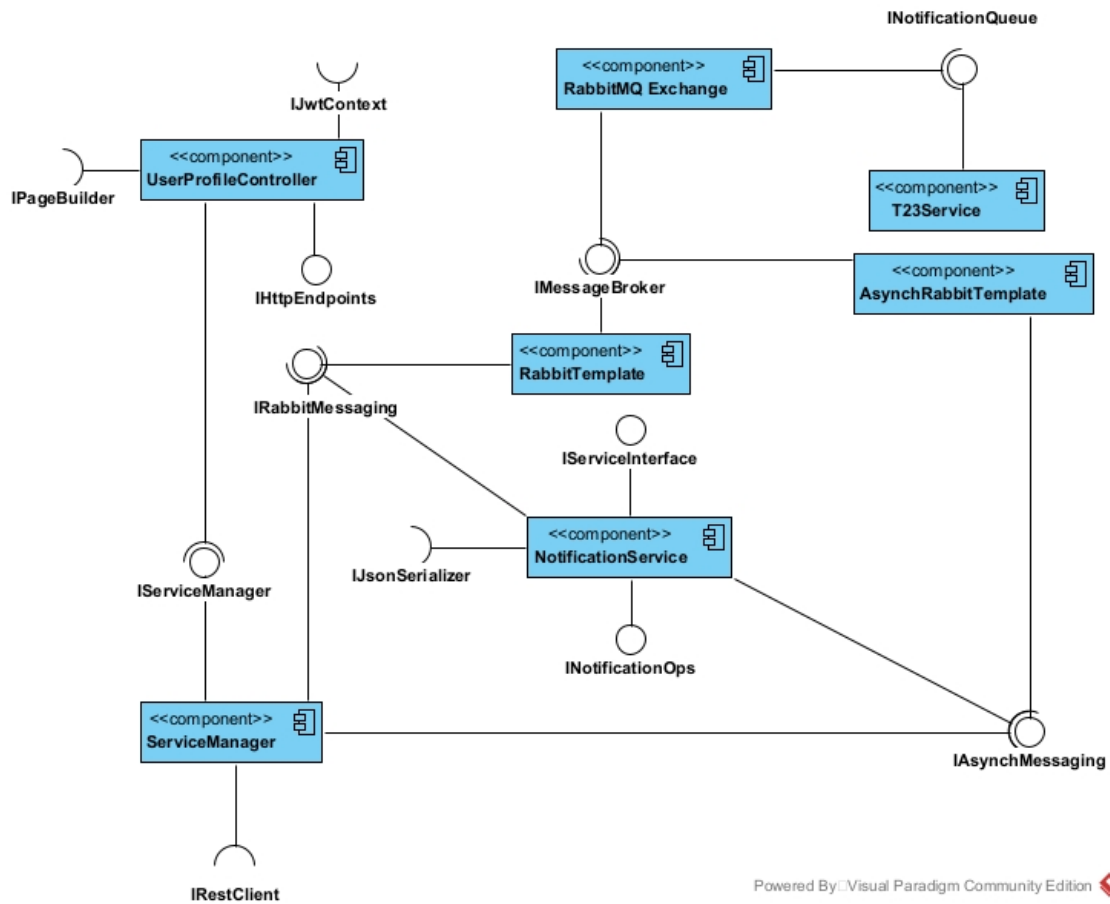


Figure 2.2: Component Diagram

Le Interfacce reali, cioè quelle che effettivamente esistono nel codice sono:

- `ServiceInterface` : rappresenta il contratto base per tutti i servizi gestiti dal Service Manager

Le Interfacce ideali, che non esistono effettivamente nel codice, ma sono state inventate a scopo rappresentativo, sono :

- `IServiceManager`: rappresenta le operazioni che il `ServiceManager` espone al controller
- `IRabbitMessaging` : `RabbitTemplate` fornisce le operazioni di invio messaggi 'fire-and-forget'
- `IAsyncMessaging` : `AsyncRabbitTemplate` fornisce le operazioni di invio messaggi con attesa risposta 'RPC'
- `IRestClient` : `RestTemplate` fornisce le operazioni HTTP REST
- `IJsonSerializer` : `ObjectMapper` fornisce le operazioni di conversione JSON per `NotificationService`, da usare in `getNotifications` per convertire la risposta Map in `NotificationResponse`
- `IMessageBroker` : rappresentante della comunicazione AMQP dei template col broker
- `INotificationQueue` : rappresenta le operazioni che T23 espone tramite coda
- `INotificationOps` : mostra le capacità specifiche del servizio notifiche (new, get, delete, update, clear)
- `IHttpEndpoints` : rappresenta gli endpoint REST che il controller espone
- `IPageBuilder` : contiene le operazioni per costruire una pagina web
- `IJwtContext` : rappresenta le operazioni per accedere al contesto di autenticazione

Chapter 3

Progettazione

3.1 Progettazione

Dalla fase di analisi, in cui sono stati definiti i casi d'uso, gli attori principali e le relazioni, si passa alla fase di progettazione, in cui si definisce come implementare questi requisiti nel sistema. Lo scopo della fase di progettazione è individuare l'architettura generale del sistema, i componenti principali, le loro responsabilità, al fine di garantire una soluzione modulare, manutenibile e scalabile. Il sistema di gamification gestisce le interazioni tra utenti, microservizi e dati persistenti, assicurando che le notifiche e i progressi siano correttamente elaborati e restituiti all'utente.

3.2 Class Diagram

In questa fase, le classi vengono dettagliate con tutti gli attributi, i loro tipi, la visibilità e i metodi completi con parametri e tipi di ritorno. Il diagramma mostra l'organizzazione del codice in package.

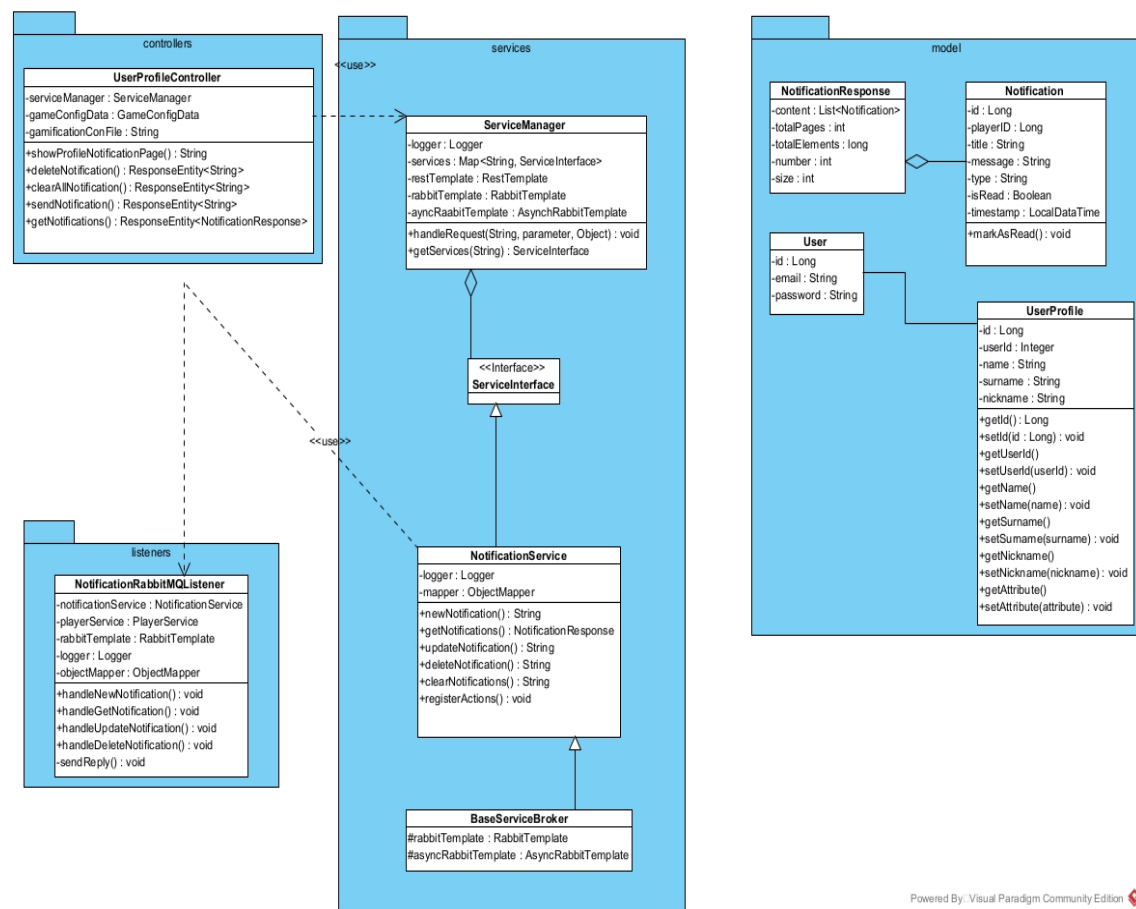


Figure 3.1: Class Diagram in fase di progettazione

3.3 Sequence Diagram

I sequence diagram di progettazione descrivono nel dettaglio le interazioni tra gli oggetti del sistema durante l'esecuzione di specifici casi d'uso. Di seguito è riportato il diagramma di sequenza per il caso d'uso showProfileNotificationPage

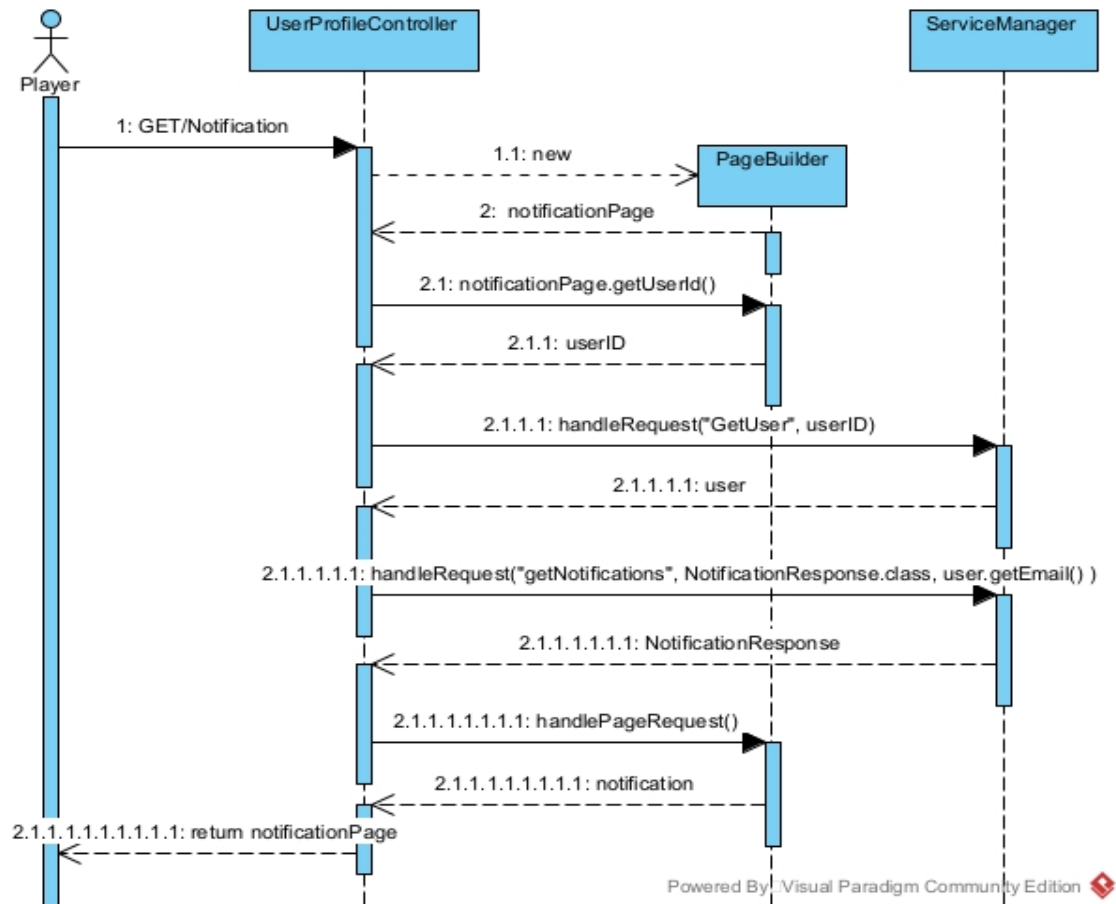


Figure 3.2: Sequence Diagram : showProfileNotificationPage

Di seguito riportato il diagramma di sequenza per il caso d'uso sendNotification

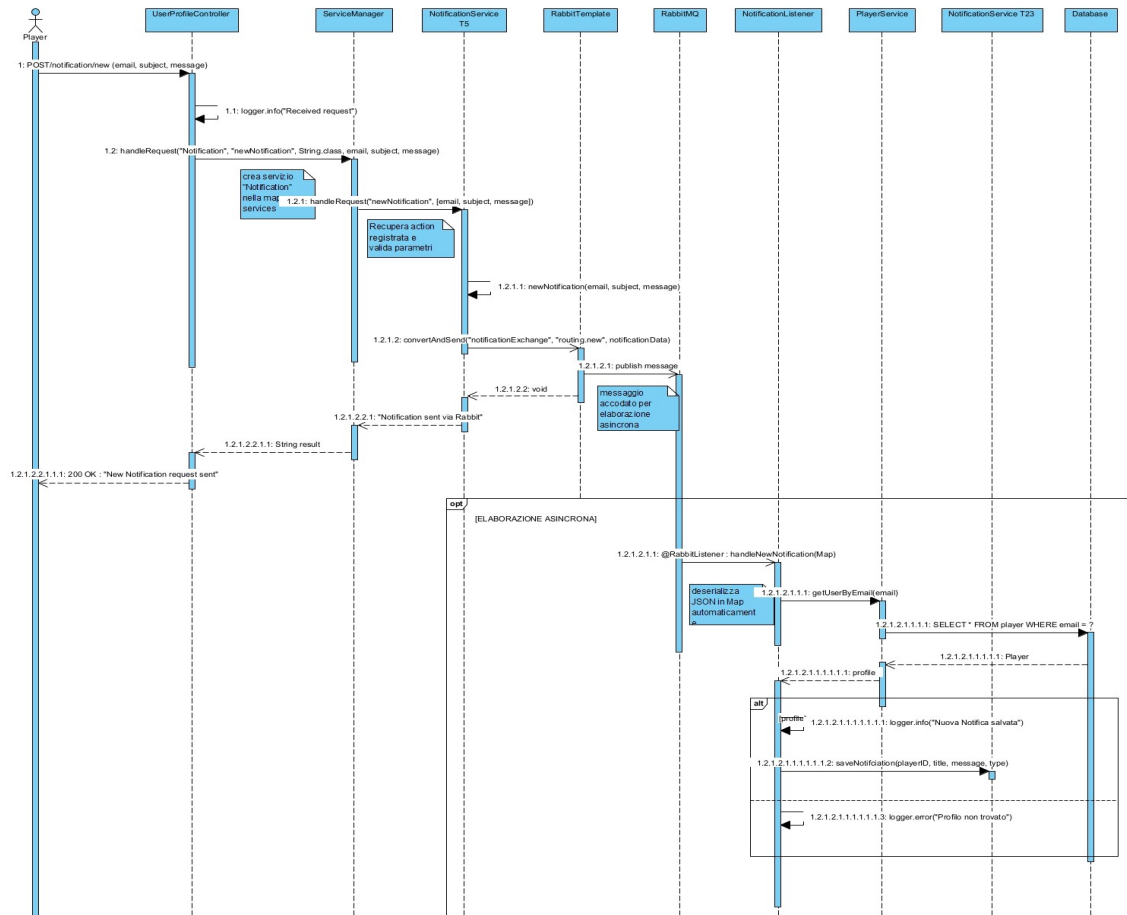


Figure 3.3: Sequence Diagram : sendNotification

Di seguito riportato il diagramma di sequenza per il caso d'uso getNotifications

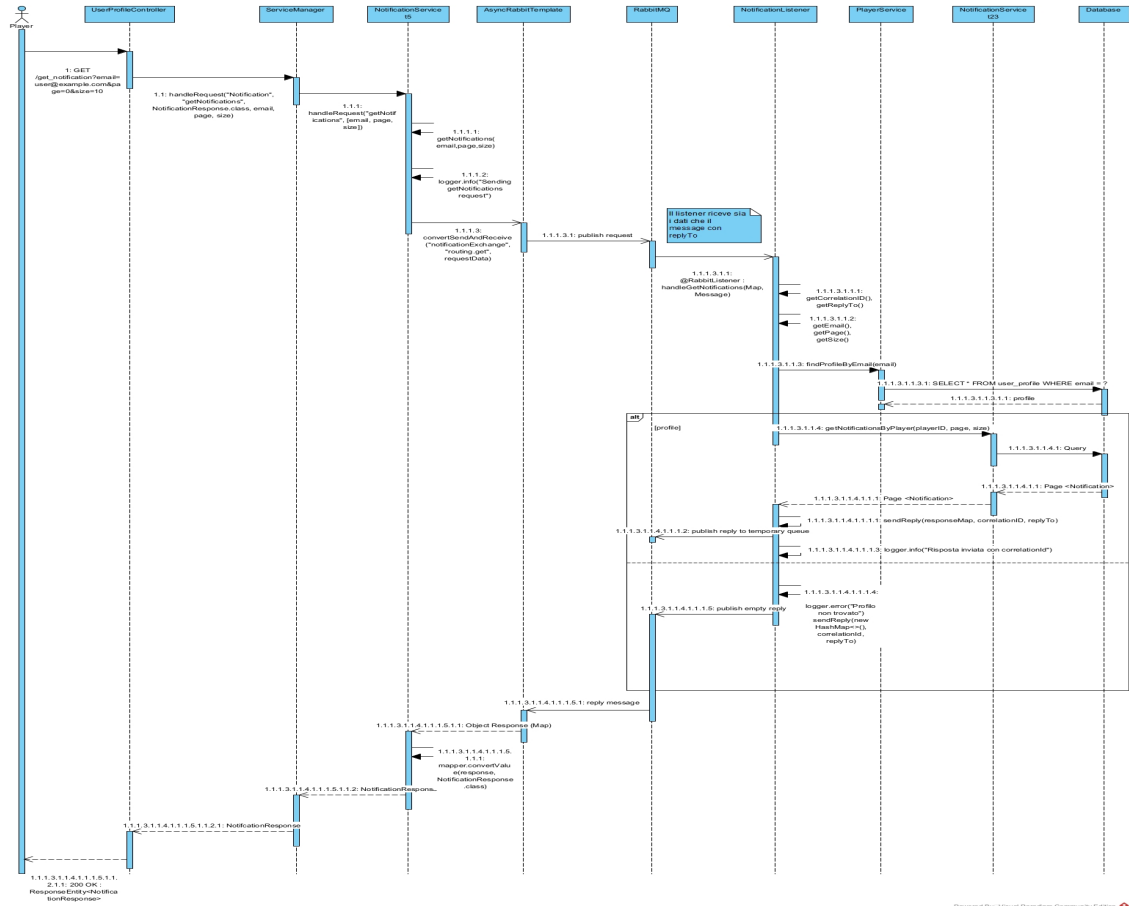


Figure 3.4: Sequence Diagram : getNotifications

3.4 Anti-Pattern

Durante la fase di progettazione della Gestione Notifiche, abbiamo adottato scelte architetturali mirate a prevenire noti Anti-Pattern tipici delle Architetture a Microservizi.

- L'anti-pattern Shared Database si verifica quando due o più Microservizi accedono direttamente al database privato di un altro servizio, causando accoppiamento stretto e perdendo il principio di autonomia. Il microservizio T23 possiede i dati delle notifiche, mentre T5 ha bisogno di leggerli. Abbiamo garantito che T5 non acceda mai al database di T23. Tutta la comunicazione avviene in modo disaccoppiato tramite il Message Broker, utilizzando il pattern Request/Reply per lo scambio di DTO (Notification-Response). Questo mantiene i dati persistenti di T23 privati, in linea con il principio dei componenti loosely coupled.
- L'anti-pattern Local Logging si verifica quando ogni servizio registra i log in modo indipendente, rendendo complessa la diagnosi dei problemi distribuiti e il tracciamento delle chiamate tra servizi. Attualmente sia T23 sia T5 registrano i propri log localmente. Quindi l'impatto riscontrato è che in caso di errore nel flusso request/reply, il tracciamento di un singolo messaggio attraverso tutti i componenti risulta estremamente difficile.

float graphicx

Chapter 4

Implementazione

Modifiche effettuate

Per evitare di appesantire la documentazione con estratti di codice estesi, riportiamo di seguito una descrizione dettagliata ma discorsiva delle principali modifiche effettuate ai microservizi e all'infrastruttura, necessarie per l'integrazione del broker RabbitMq nel sistema. L'introduzione del broker di messaggistica ha richiesto interventi su più moduli.

Modifiche al microservizio T23

All'interno del microservizio T23 è stata introdotta una nuova classe listener deputata alla ricezione dei messaggi inviati dal broker. Questa classe gestisce :

- la sottoscrizione alla coda;
- la deserializzazione dei messaggi;
- l'invocazione della logica applicativa per la creazione, consultazione o cancellazione delle notifiche

T23 è inoltre il microservizio che possiede l'accesso diretto al database tramite la classe NotificationRepository. Ciò ci consente di :

- recuperare i dati dell'utente a partire dall'email fornita;
- generare, aggiornare o eliminare notifiche persistendole nel database;
- rispondere eventualmente a T5 solamente per le operazioni di *get*, per cui si predisponde un pattern RPC e dove è necessario resituire informazioni sulla notifica richiesta. In tutti gli altri casi, T23 elabora autonomamente il messaggio senza produrre una risposta.

Modifiche al microservizio T5

Nel microservizio T5 è stata rivista la logica dei metodi dedicati alla gestione delle notifiche, in modo da demandare completamente al broker la comunicazione verso T23.

T, infatti, non effettua più chiamate dirette al database, poichè non possiede alcun accesso ai dati persistenti. Tutte le informazioni, come l'email dell'utente o l'identificativo vengono prelevate da parametri della richiesta.

Successivamente, T5 si limita ad impacchettare tali informazioni in un messaggio AMQP e ad inviarlo all'exchange gestito dal broker. Spetta poi a T23 eseguire le operazioni richieste, essendo l'unico modulo con accesso ai dati.

Modulo Broker

Infine, è stato introdotto un nuovo modulo dedicato alla configurazione broker. Questo modulo contiene:

- il file di configurazione dell'exchange
- la definizione delle code
- i binding necessari per collegare le code all'exchange

Tale configurazione garantisce che all'avvio dell'infrastruttura le componenti di messaggistica siano correttamente predisposte.

Inoltre, è stato necessario assicurare che il broker venga avviato prima dei microservizi che dipendono da esso. In caso contrario, i servizi tenterebbero la connessione verso un broker non ancora disponibile, causando errori e arresti automatici dei moduli T5 e T23.

4.1 Creazione di un nuovo servizio nel modulo T5

La necessità di rendere il codice più leggibile ci ha portati a estrarre i metodi che erano presenti nel modulo T23Service relativi alle notifiche e di creare una nuova classe all'interno del package T5-G2/t5/src/main/java/com/g2/interfaces denominata *NotificationService*

Di seguito riportata una rappresentazione delle classi *prima* della modifica :

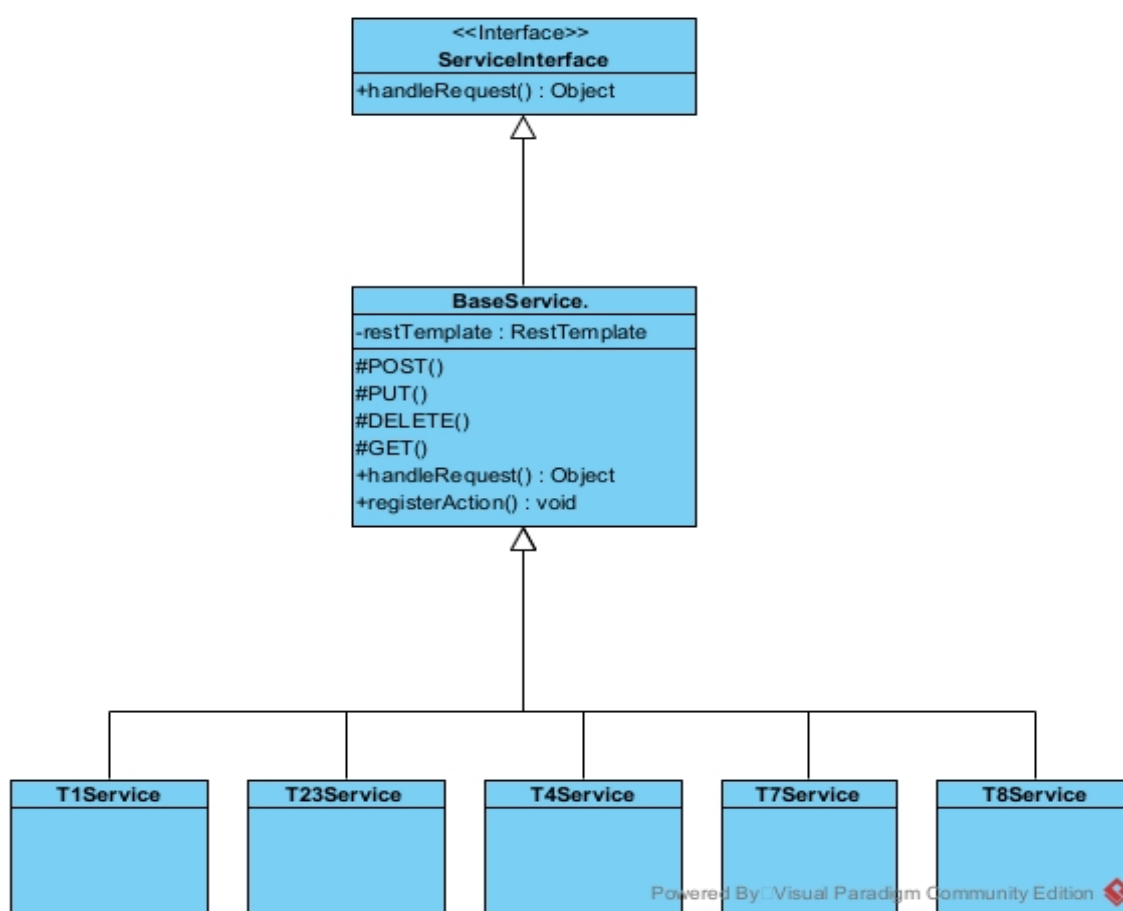


Figure 4.1: Class Diagram prima della modifica

Di seguito riportata una rappresentazione delle classi *dopo* la modifica :

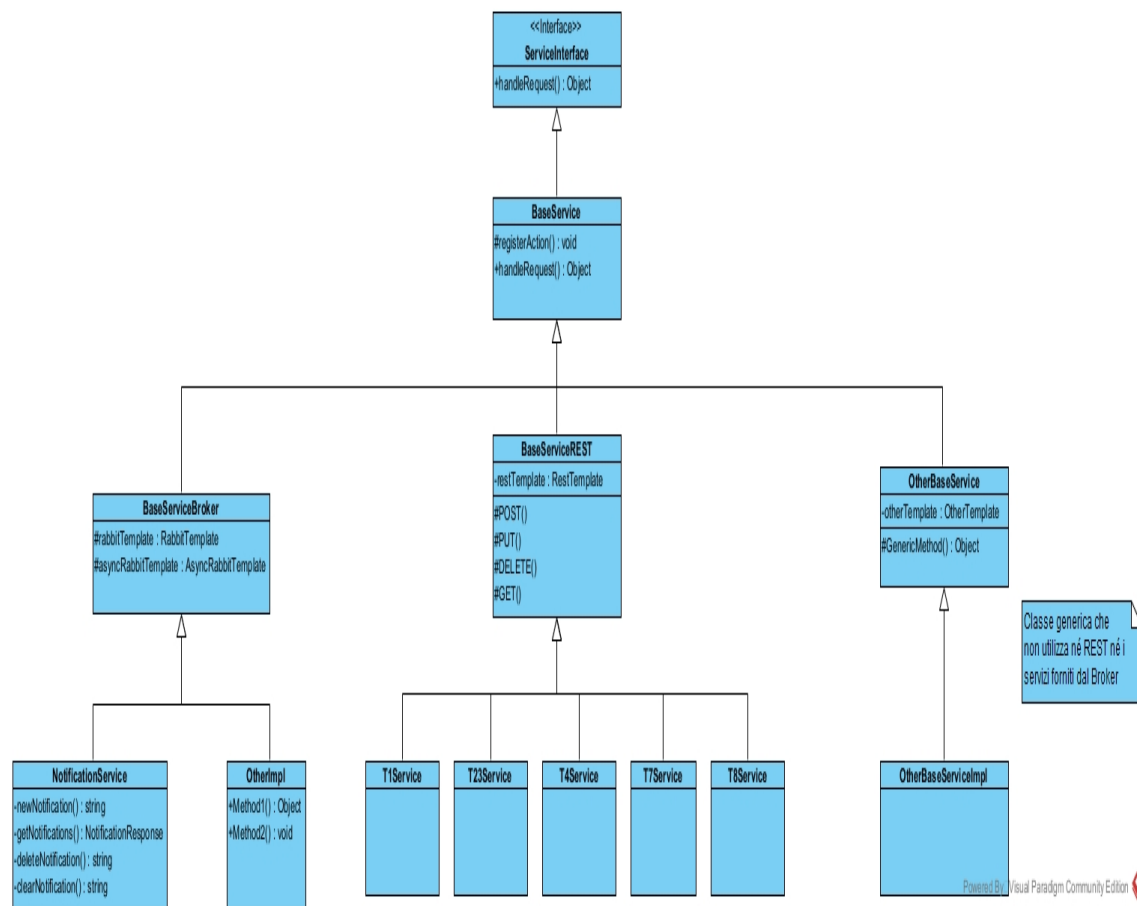


Figure 4.2: Class Diagram dopo la modifica

4.2 Deployment Diagram

Il diagramma di deployment è uno dei diagrammi strutturali il cui scopo è rappresentare l'architettura fisica del sistema, mostrando come i componenti software sono distribuiti e su quali nodi vengono eseguiti. È particolarmente utile nei sistemi distribuiti, a microservizi, client-server, oppure quando esistono più ambienti fisici come :

- server applicativi
- broker di messaggi
- database
- container Docker
- dispositivi client

Gli elementi principali del diagramma sono:

- Nodo : elemento fisico che partecipa all'esecuzione del sistema
- Artefatto : file fisico (.jar, libreria, file di configurazione)
- Device Node : hardware fisico
- Execution Environment Node : ambiente software che ospita processi
- Percorso di comunicazione : i nodi nel diagramma vengono collegati tramite link di comunicazione che rappresentano protocolli (HTTP, AMQP), flussi di dati o scambio di messaggi

Nel diagramma riportato di seguito sono stati rappresentati tutti i container in un unico spazio docker host. Ciascuno con le rispettive porte di accesso e tipo di collegamento. Si pone attenzione sulla comunicazione che il client effettua col *solo* modulo T5 e sulla comunicazione che il database effettua col *solo* modulo T23.

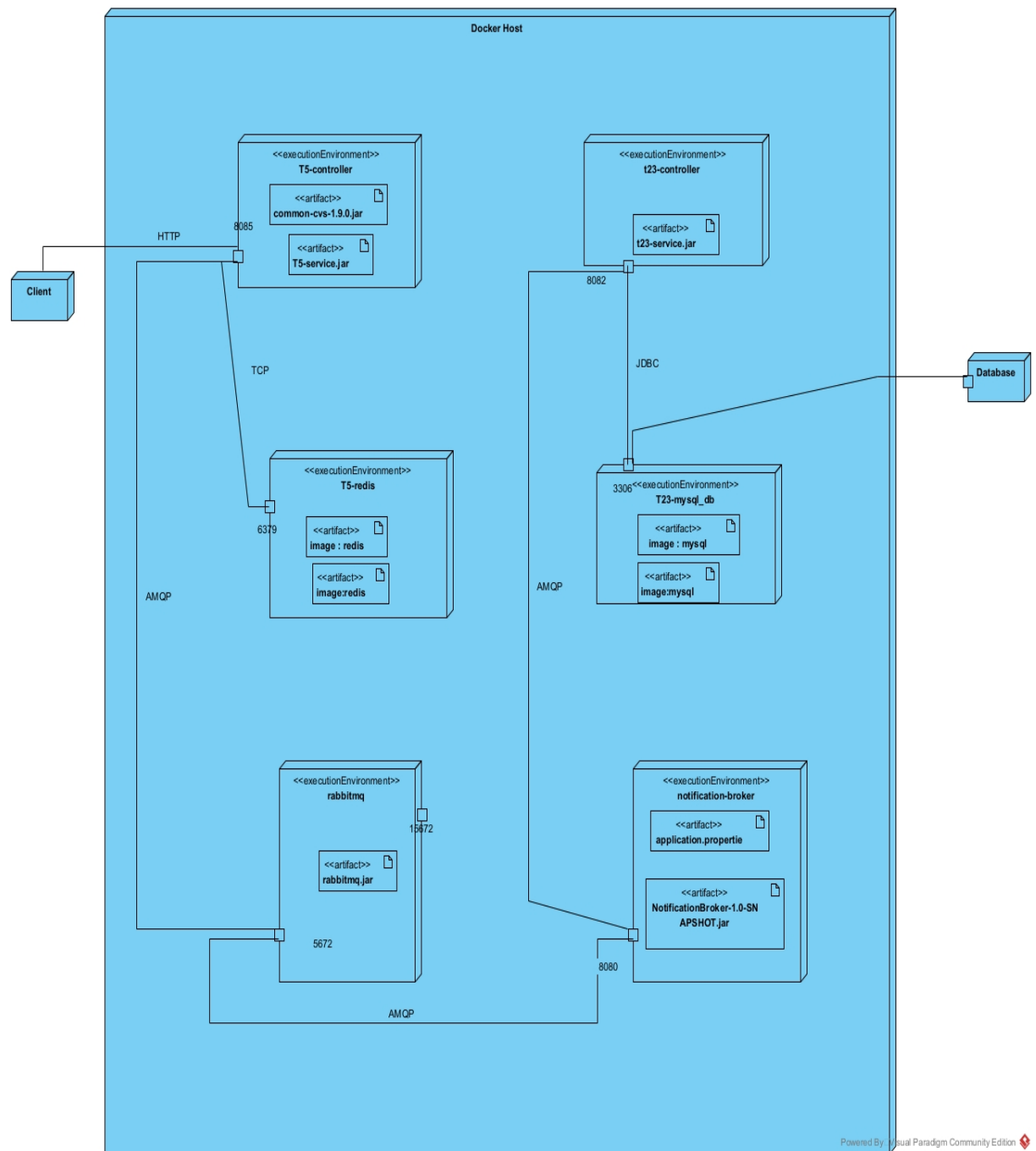


Figure 4.3: Deployment Diagram finale dei moduli T23, T5 e NotificationBroker

4.3 Problematiche riscontrate

Nel corso dell'integrazione sono emerse alcune criticità, principalmente dovute a inconsistenze tra gli endpoint documentati e quelli realmente esposti dai vari moduli. Tra i problemi principali segnaliamo :

- Mismatch negli endpoint tra T5 e API/UI Gateway.
Alcuni endpoint utilizzati da T5 non coincidevano con quelli previsti dall'API Gateway. Ad esempio:
 - l'UI Gateway inoltra automaticamente tutte le richieste verso T23,
 - mentre T23 si aspettava le operazioni sulle notifiche all'interno del path */Notification/newNotification*

Questo disallineamento ha causato inizialmente il fallimento delle richieste. Il problema è stato risolto aggiornando la configurazione del gateway per indirizzare correttamente le richieste effettivamente utilizzate dal microservizio T23.

- Incoerenze negli endpoint relativi agli utenti.
Un altro problema riguardava la gestione delle risorse utente. Molti metodi utilizzavano prefissi del tipo */student/method* , mentre T23 si aspettava endpoint del tipo: */player/method*. Questa discrepanza impediva la corretta risoluzione degli utenti e comprometteva l'invio della notifica indirizzata ad un singolo destinatario. La questione è stata risolta riallineando i prefissi e correggendo la logica dei servizi.
- Il modulo T23, in particolare il database, ha sempre riscontrato problemi nell'esecuzione. Per superare ciò è stato necessario rimuovere campi all'interno del docker-compose: commentando l'entry point del database.

4.3.1 Risultato finale

Al termine delle modifiche :

- T5 può ora inviare notifiche generiche al sistema senza interagire direttamente con il database;
- T23 gestisce la parte di business e di persistenza, delegando completamente la comunicazione a RabbitMQ;
- il nuovo modulo broker garantisce la disponibilità delle code e dell'exchange al momento dell'avvio del sistema.

È stato effettuato un controllo dei file *pom* dei moduli interessati, dal quale è emersa l'assenza della dipendenza relativa a RabbitMQ. Tale mancanza è stata corretta includendo la dipendenza necessaria nei moduli che interagiscono con il broker. Per quanto riguarda la configurazione dei DockerFile, è stato utilizzato il parametro *depends-on*, che consente di garantire che l'applicazione incaricata della configurazione del broker (in particolare la creazione e gestione delle code) venga avviata solo successivamente all'avvio del servizio RabbitMQ. Questo approccio assicura il corretto ordine di inizializzazione dei container ed evita problemi di connessione al broker non ancora disponibile.

Relativamente ai file *application.properties*, nel modulo dedicato al broker vengono specificati l'host di RabbitMQ, la porta di comunicazione e le credenziali di accesso (guest, guest). Negli altri moduli dell'applicazione, che non richiedono l'accesso diretto al broker per operazioni di configurazione, vengono invece indicati esclusivamente il nome dell'host e la porta, omettendo le credenziali.

4.4 Recap: Moduli modificati

Modulo / Classe	Descrizione della modifica
T5 – UserProfileController	Sono stati inseriti i metodi <i>getNotifications</i> , <i>sendNotification</i> , <i>deleteNotification</i> e <i>clearAllNotifications</i> per la gestione delle notifiche lato UI.
T5 – GuiController	modifica per la generazione di una notifica quando si accede alla modalità <i>partita singola</i>
T5 – interfaces – T23Service	Eliminazione dei metodi relativi alle notifiche, successivamente estratti e riorganizzati in una nuova classe dedicata.
T5 – interfaces – NotificationService	Nuova classe introdotta per contenere la logica di gestione delle notifiche precedentemente presente in <i>T23Service</i> .
T5 – interfaces – BaseServiceREST	Nuova classe contenente le chiamate REST eliminate dalla classe <i>BaseService</i> , al fine di separare la comunicazione sincrona da quella asincrona.
T5 – interfaces – BaseServiceBroker	Classe introdotta per incapsulare i template di RabbitMQ e gestire l'invio dei messaggi AMQP verso il broker.
NotificationBroker – Broker-Configuration	presente la configurazione del binding e la creazione delle code, la gestione della serializzazione dei messaggi e il RabbitAdmin per la creazione automatica all'avvio di tutte le code ed exchange
NotificationBroker – Application	effettua l'inizializzazione del broker all'avvio dell'applicazione
T23 – config – RabbitMQ-Config	Nuova classe per la configurazione di RabbitMQ nel servizio T23. Definisce i bean necessari per la corretta serializzazione e deserializzazione dei messaggi.
T23 – service – NotificationRabbitMQListener	Classe contenente il listener che si mette in ascolto sulle code RabbitMQ per gestire le operazioni relative alle notifiche. Ogni metodo annotato con <code>@RabbitListener</code> rappresenta un consumer associato a una specifica coda.
T23 – service – NotificationService	Classe che incorpora la classe <i>NotificationRepository</i> per l'interfacciamento con il database che logicamente ingloba la classe DAO nelle funzionalità aggiungendoci le proprietà di paginazione dei dati.
Api/UiGateway	sono state modificate le routes per renderle compatibili con le chiamate per la richiesta della pagina

Table 4.1: Riepilogo dei moduli e delle classi modificate o introdotte

Chapter 5

Testing

5.1 Test funzionale

test case id	descrizione	pre-condizioni	input	output attesi	post-condizioni	output tenuti	ot-	esito
1	richiesta new notification	l'email deve esistere	email="test@email.com", subject="Oggetto Test",message="Messaggio"	inserimento della notifica	la notifica è caricata nel db	visualizzazione della notifica a schermo		PASS
2	richiesta new notification	l'email deve esistere	email="test@email.com", subject=null,message="Messaggio"	inserimento della notifica	la notifica è caricata nel db	visualizzazione della notifica a schermo		PASS
3	richiesta new notification	l'email deve esistere	email="test@email.com", subject="Oggetto Test",message= null	inserimento della notifica	la notifica è caricata nel db	visualizzazione della notifica a schermo		PASS
4	richiesta delete notification	l'email deve esistere	email="test@email.com", id= non esistente	cancellazione della notifica	la notifica è eliminata dal db	cancellazione della notifica		PASS
5	richiesta delete notification di una notifica non esistente	l'email deve esistere	email="test@NonEsiste.com", id=12345	invio di notifica eliminata ugualmente	la notifica continua a non esistere nel db	invio di notifica eliminata		PASS
6	richiesta clear notification	l'email deve esistere	email="test@NonEsiste.com"	invio notifica di non esistenza email	l'email non esiste	invio notifica di non esistenza email		PASS

Table 5.1: Test funzionali

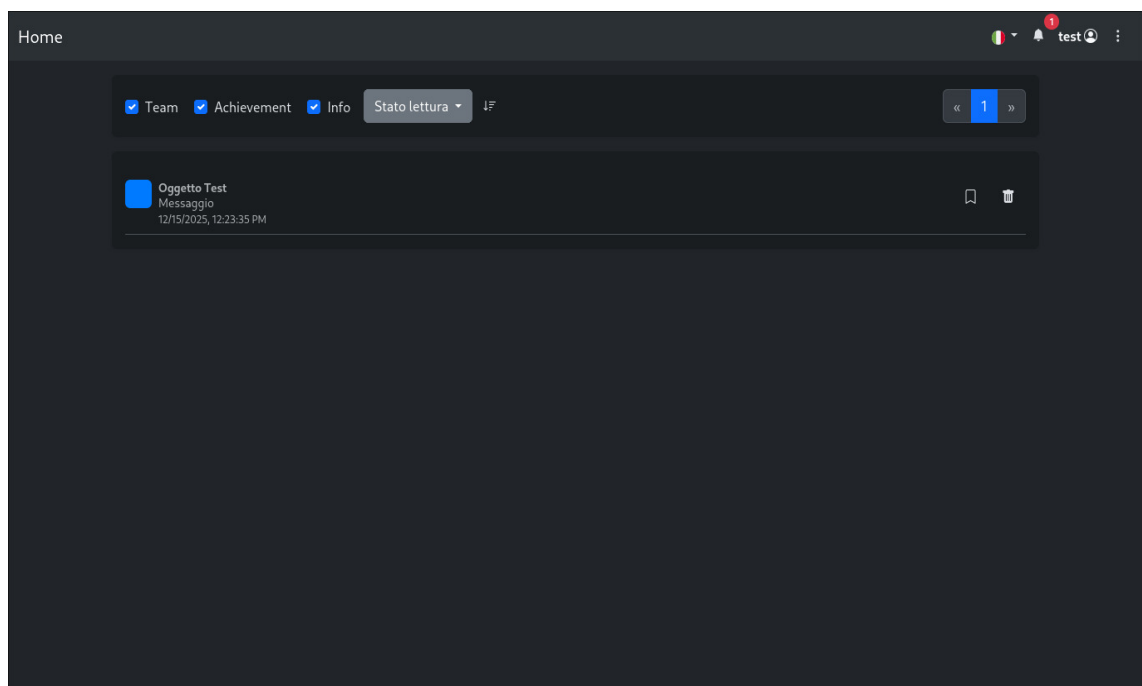
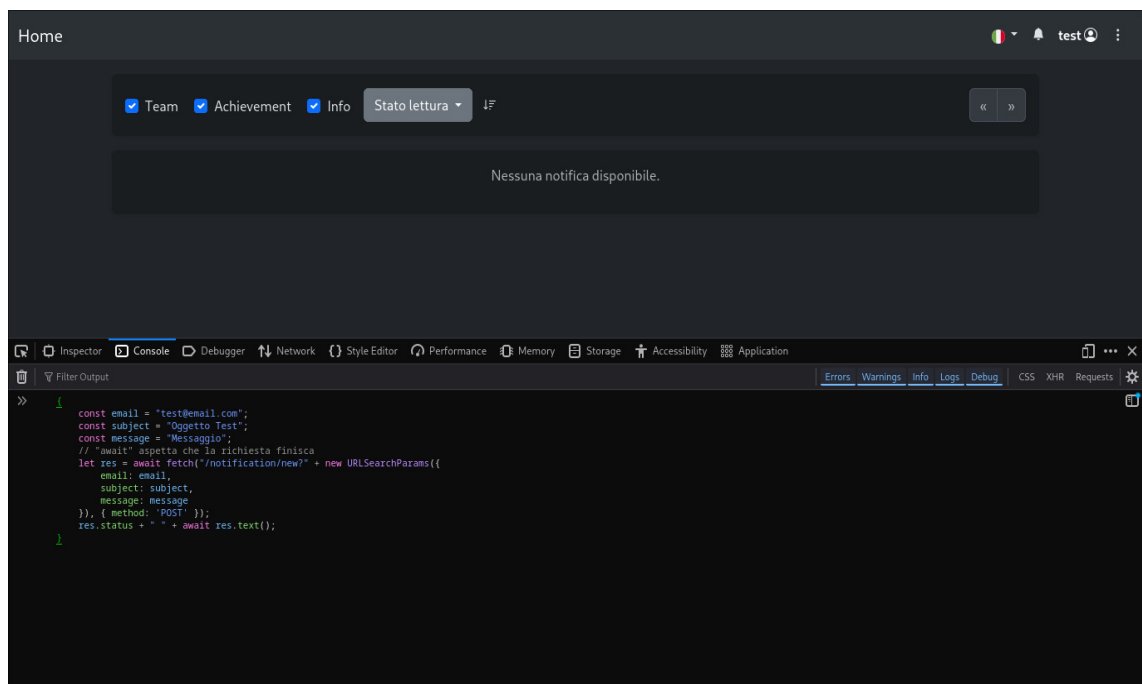


Figure 5.1: Test case : 1

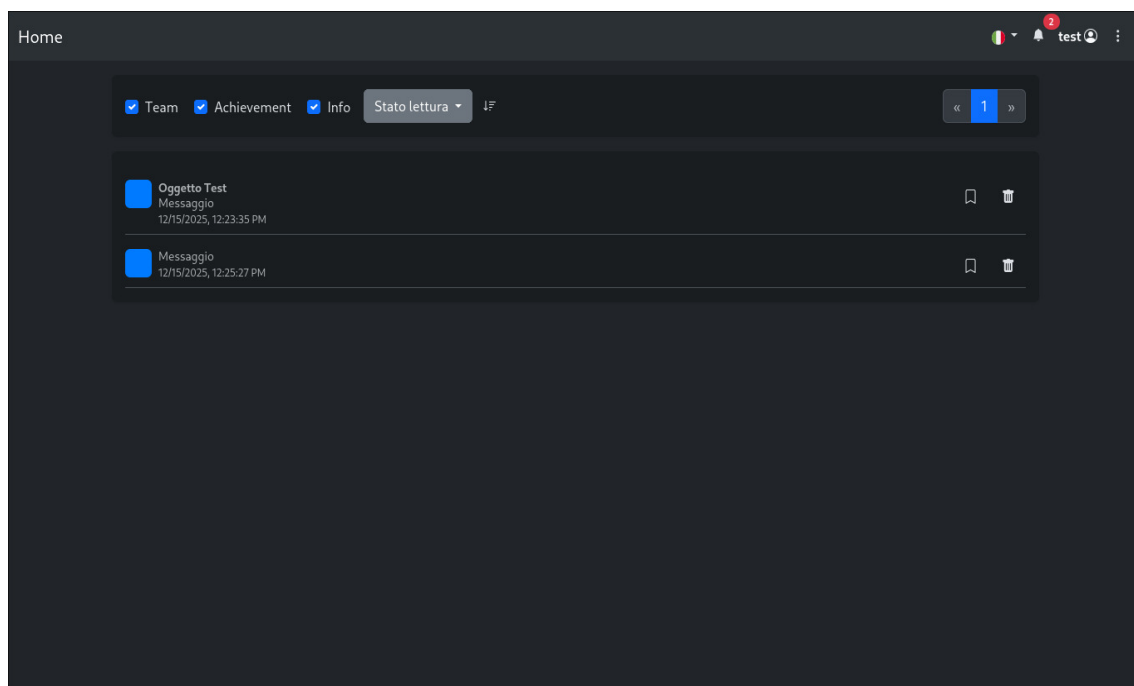
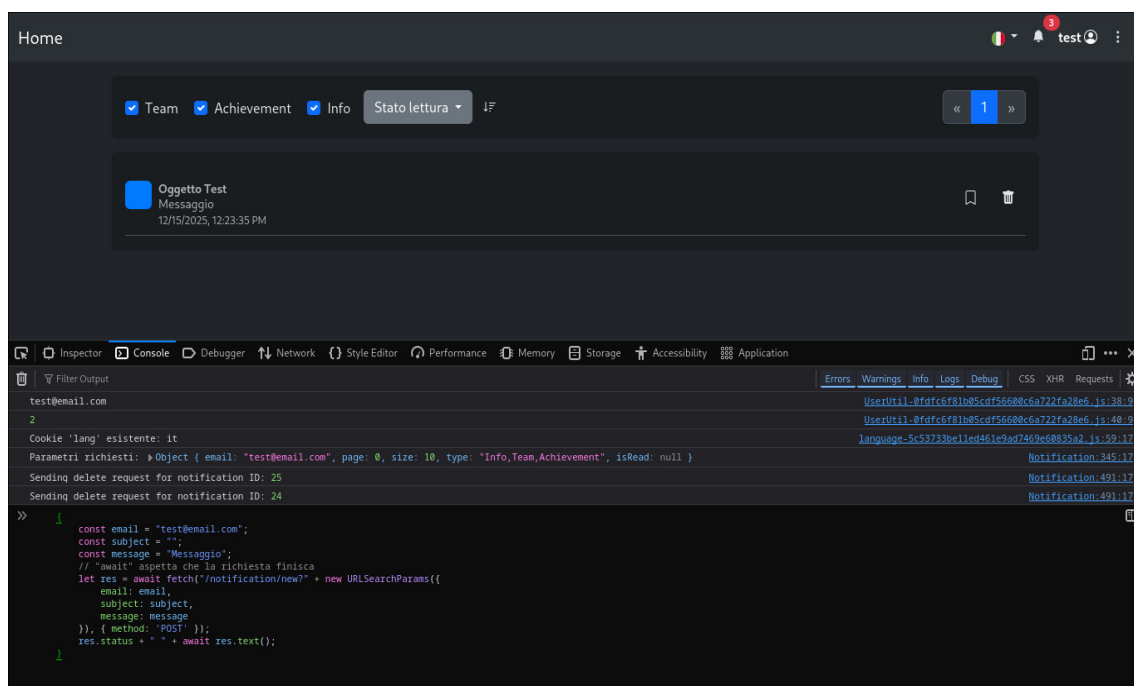


Figure 5.2: Test case : 2

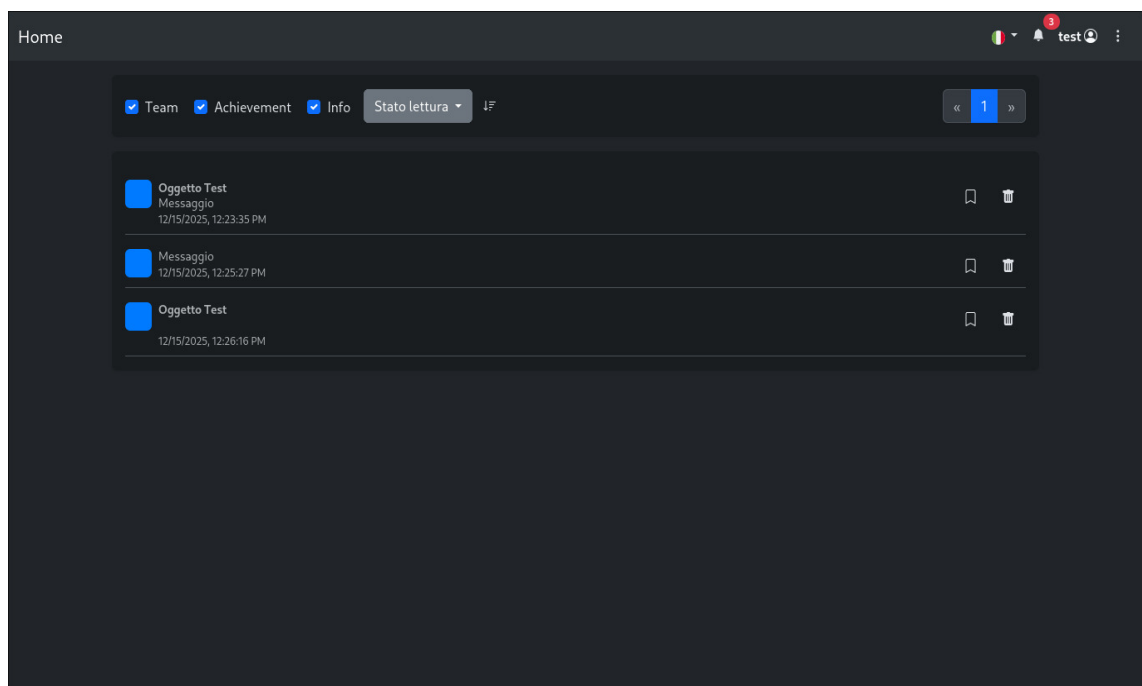
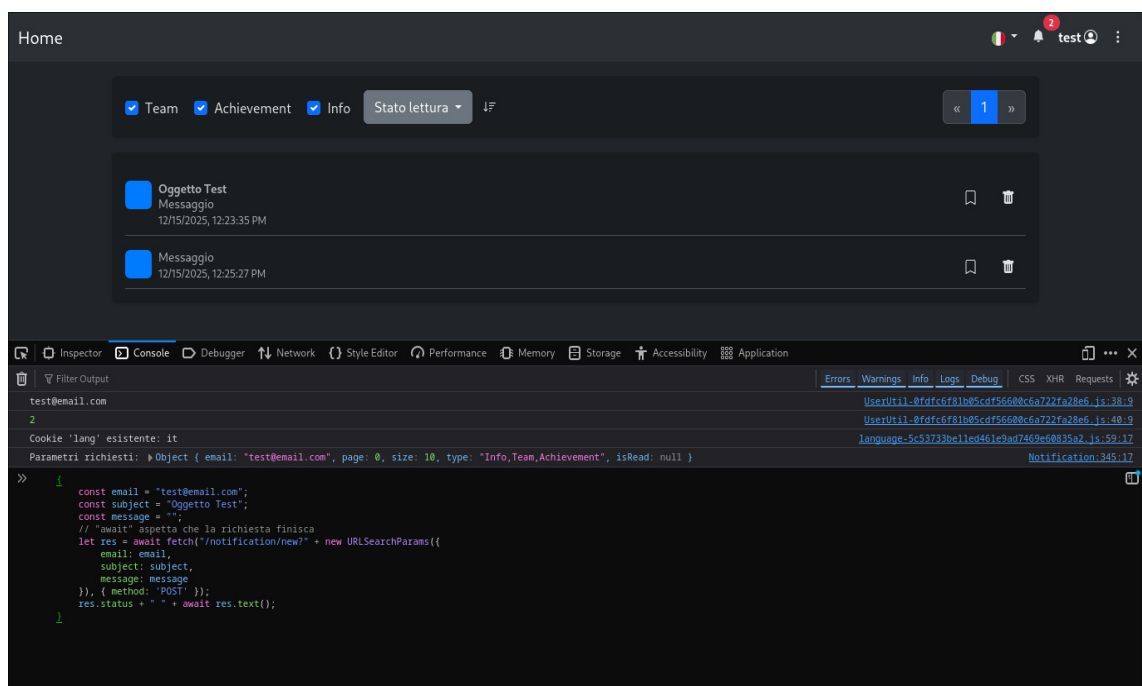


Figure 5.3: Test case : 3

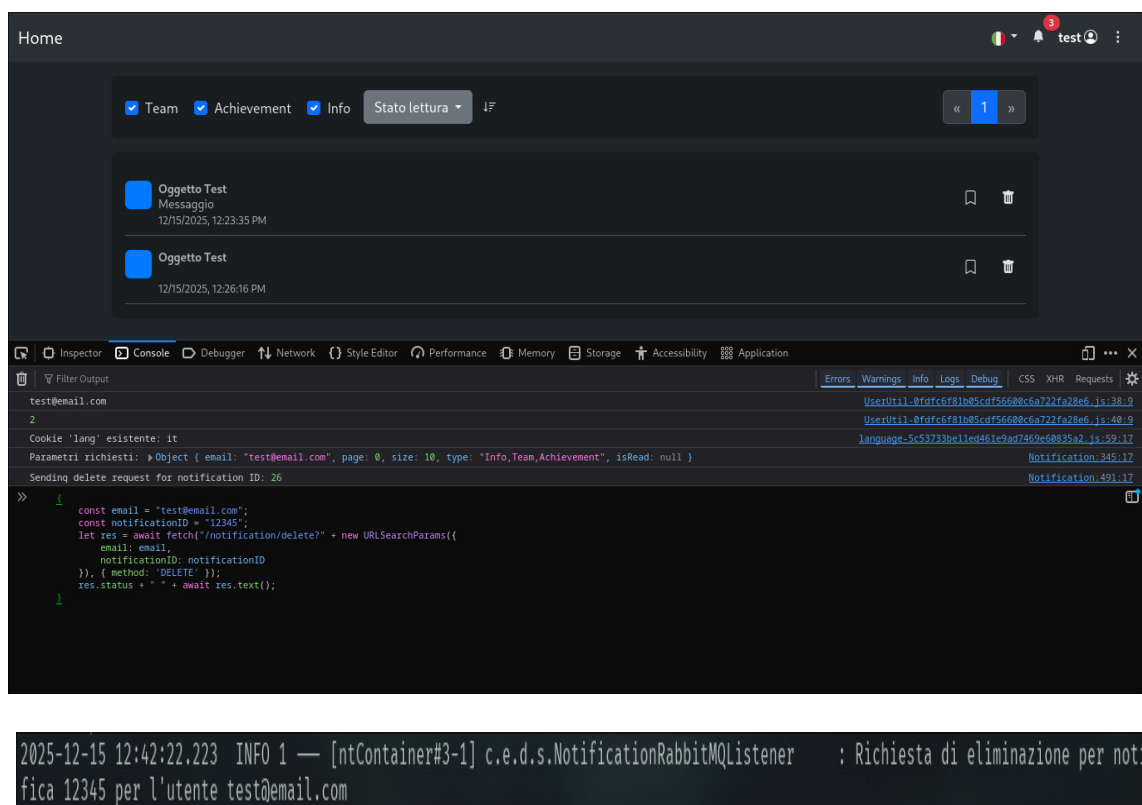


Figure 5.4: Test case : 4

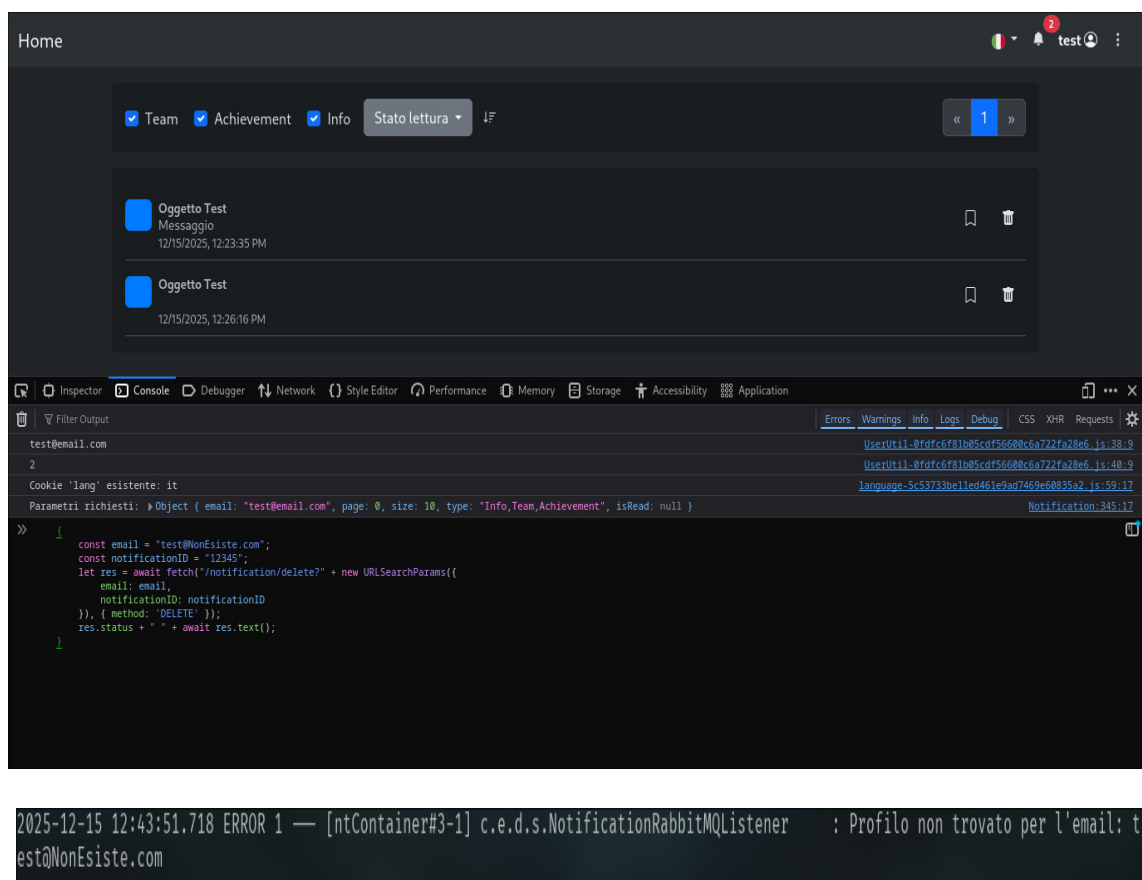


Figure 5.5: Test case : 5

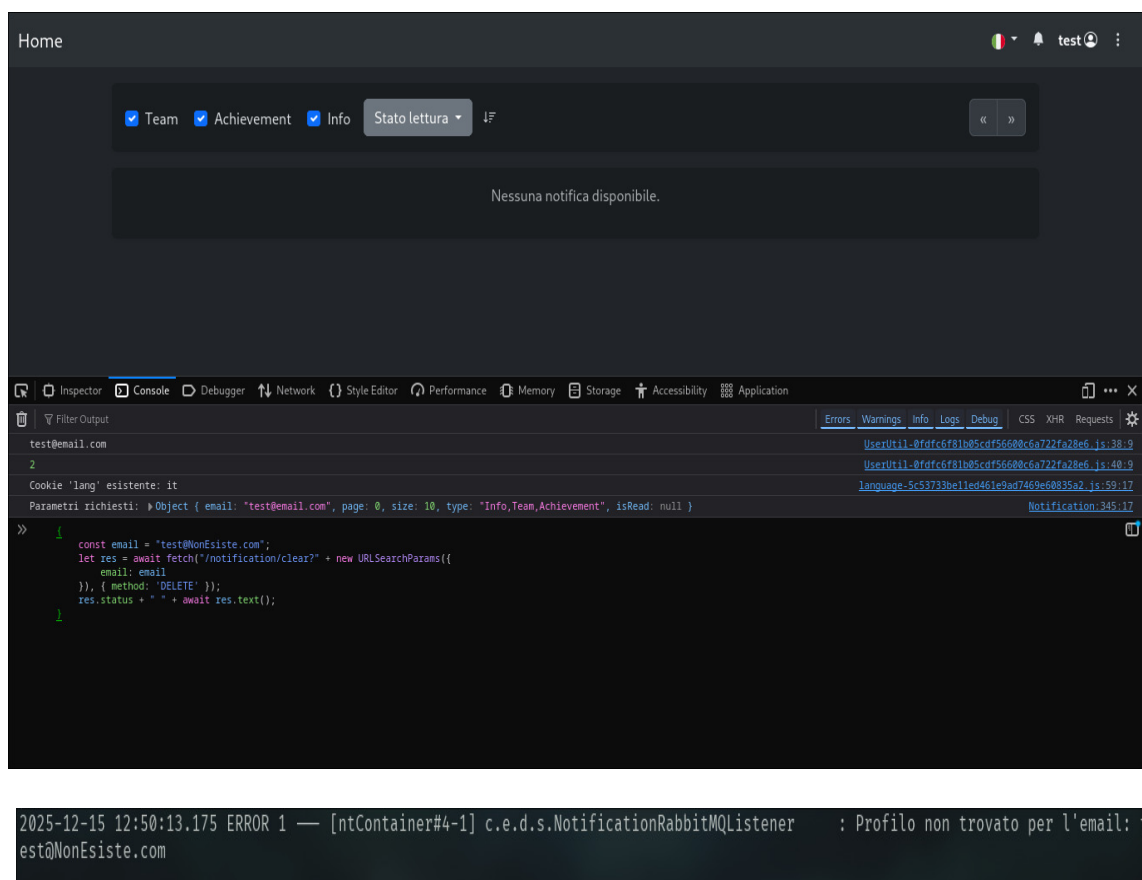


Figure 5.6: Test case : 6

Chapter 6

Conclusioni

Il presente progetto ha affrontato con successo l'implementazione del requisito R7, introducendo un sistema generalizzato di notifiche nell'architettura a microservizi dell'applicazione Educational Game for Man vs Automated Testing Tools. L'approccio adottato ha permesso di raggiungere gli obiettivi prefissati, garantendo flessibilità, scalabilità ed affidabilità nella gestione delle notifiche tra i diversi moduli del sistema.

6.1 Risultati Raggiunti

L'introduzione di RabbitMQ come message broker ha rappresentato la scelta architetturale centrale del progetto, permettendo di sostituire la comunicazione sincrona REST tra i moduli T5 e T23 con una comunicazione asincrona basata su messaggistica. Questa modifica ha portato benefici significativi:

- Dissaccoppiamento: I moduli T5 e T23 ora comunicano attraverso il broker, migliorando la manutenibilità del sistema.
- Riduzione della latenza: L'adozione del pattern Fire-and-Forget per le operazioni di creazione, aggiornamento ed eliminazione delle notifiche ha permesso di restituire risposte immediate al client, senza attendere l'effettiva persistenza dei dati
- Affidabilità: La persistenza dei messaggi garantita da RabbitMQ assicura che nessuna notifica venga persa, anche in caso di momentanea indisponibilità del modulo T23
- Scalabilità: L'architettura a code permette di gestire un elevato numero di richieste concorrenti, distribuendo il carico tra più istanze consumer quando necessario.

6.1.1 Scelte progettuali

L'adozione di due pattern distinti di comunicazione (Fire-and-Forget per le operazioni senza risposta e RPC con code temporanee per le operazioni che richiedono un risultato) ha dimostrato di essere una soluzione equilibrata, capace di ottimizzare le performance mantenendo al contempo la necessaria sincronizzazione quando richiesta.

La riorganizzazione del codice nel modulo T5, con l'estrazione della logica delle notifiche dalla classe T23Service verso la nuova classe NotificationService, ha migliorato significativamente la leggibilità e la coesione del codice, garantendo una separazione delle responsabilità.

La scelta di mantenere database privati per ciascun microservizio è stata rispettata: T23 rimane l'unico modulo con accesso diretto al database delle notifiche, mentre T5 si limita a inviare messaggi tramite broker. Questo approccio garantisce l'indipendenza dei servizi e facilita eventuali evoluzioni future dello schema dati.

6.1.2 Sviluppi Futuri

Per gli sviluppi futuri, viste le scarse prestazioni dell'applicazione in caso di più utenti si potrebbero implementare, per quanto riguarda il deploy, dei meccanismi di load balancing che possano migliorare la gestione del traffico e l'esposizione dei container in rete.

Analizzando però il codice, si è visto come potrebbero sorgere dei problemi a causa dell'utilizzo di '-' negli url per il raggiungimento di moduli necessari al funzionamento.

Ma una volta modificato ciò, le application.properties e creato dei file di config e modificato i Service si potrebbe creare un file .yaml dove vengano specificati:

- Deployment per T5-G2: Con varie repliche.
- Service per T5-G2: Di tipo LoadBalancer per esporre il servizio all'esterno.
- Deployment per T23-G1: Con varie repliche.
- Service per T23-G1: Di tipo ClusterIP per la comunicazione interna.
- Deployment per RabbitMQ: Con 1 replica.
- Service per RabbitMQ: Di tipo ClusterIP.

Il tutto sfruttando che RabbitMQ sia un servizio Stateful (mantiene lo stato delle code).

Una singola istanza di RabbitMQ ben configurata può gestire decine di migliaia di messaggi al secondo. Il "collo di bottiglia" è quasi sempre chi processa il messaggio (T23) a causa degli accessi ai dati, non chi lo trasporta (RabbitMQ).

6.1.3 Considerazioni Finali

Questo progetto ha rappresentato un'importante opportunità di applicare concetti avanzati di architettura software in un contesto reale. Il team ha acquisito competenze pratiche nell'uso di tecnologie moderne (Spring Boot, RabbitMQ, Docker), nell'applicazione di pattern architetturali (Fire-and-Forget, RPC, Broker Pattern) e nella gestione di progetti complessi attraverso metodologie Agile.